

Intent-Based Developer Platform

Author: Vinay Vittal

AI Helper and Tools: Claude Code, ChatGPT, Gemini, Mermaid, VS Code

Date Submitted: 5/7/2026

Background	1
Product Rationale	1
Customer Problem	2
Problem Statement	2
Product Vision	2
North Star Metrics	3
Segmentation	3
Pain Points	4
Developer Journey	4
Solution Prioritization	5
Solution Approach	6
Build vs Buy vs Partner	8
Product Requirements	10
Roadmap	11
MVP - Launch in 1 month w/ 4-5 engineers	13
Open Beta - Launch in 3 months	14
GA - Launch in 6 months	14
GTM Strategy	14
FAQ	14
APPENDIX	16
Exhibit A	16
Exhibit B	16
Exhibit C	17
Exhibit D	17

Background

The DevEx team at XYZ, Inc. is responsible for providing “the foundational infrastructure that enables every engineer in the company to build, deploy, and scale world-class products.” [Case Study Instructions].

Product Rationale

Developer workflows have massively shifted toward natural language in large tech companies. Engineers increasingly create code by describing intent to AI coding assistants rather than typing them. Deployment should follow the same paradigm. The intent-based agent eliminates

the friction of context-switching from conversational coding to manual YAML editing (or HELM template syntax). Developers describe deployment requirements the same way they describe code. For example - "I am deploying a Tier-2 Python service that needs to be accessible in North America and handle 1k RPS". The agent then generates validated configurations. Natural language deployment is the natural progression of natural language development. The DevEx team is uniquely positioned to build this because of their infrastructure expertise, visibility into deployment patterns, and mandate to reduce friction in the path to production.

Customer Problem

"Currently, XYZ's platform consists of several mature but disconnected internal services:

- Provisioning: Setting up cloud resources, clusters, and namespaces.
- Security Compliance: Managing secrets, IAM roles, and vulnerability scanning.
- Observability: Configuring logs, metrics, and alerting thresholds.
- Traffic Management: Setting up load balancers, DNS, and routing rules.

To launch a simple microservice, a developer must manually "glue" these services together by writing thousands of lines of boilerplate configuration (e.g., YAML, Terraform)."

Problem Statement

Developers spend a disproportionate amount of time manually configuring services that leads to slower speed to market

We will be able to determine which part of the path to production is the most broken and causing the most pain for customers, by tracking the following signals.

- Broad Strokes Signals
 - Surveys
 - CSAT score
 - NPS
- Higher Fidelity Signals
 - Customer interactions
 - Volume of support tickets tagged "deployment" or "Kubernetes" etc.
 - Volume and topic of Slack messages to the Platform team
 - Avg time from code complete to successful deployment
 - Volume and reason for deployment rollbacks
 - Number of lines of manually written config for deployment
 - Access and approval request volume between code complete and deployment

Product Vision

Note to reader: I have reframed the product vision below to be solution agnostic

Empower developers to productionize services faster and more easily

North Star Metrics

Metric 1: Average number of manually written lines of configurations per developer/service in production, MoM

Metric 2: Average infrastructure cost per service per tier, MoM

Metric 3: Average time taken to deploy a service to production, MoM

Counter Metric 1: Average number of deployment related support tickets, MoM

Counter Metric 2: Average number of services that successfully deploy, MoM

Counter Metric 3: Average number of prompts per developer, MoM

Segmentation

Note to reader: Several assumptions have been made in this section

The table below shows 5 reasonable ways to segment our market. Only one segmentation variable (bolded) has been further classified by using criteria such as TAM, unserved need and alignment. The evaluation is based on a 3 point scale.

Segmentation Variable	Segments	Total Addressable Market (TAM)	Unserved Need	Alignment w/ Product Vision	Total
Service Type	Containerized	3	3	3	9
	Serverless	2	1	2	5
	Others, Custom	1	3	2	6
Service Peak RPS	Up to 1,000 per service				
Service Region	North America for initial launch				
Customer Team Size	Any				
Customer Team Skill	SDLC				

We will choose **Service Type** requirement as the segmentation variable. Most services that need infrastructure are containerized so addressing this segment will be the most impactful. The needs of services that are deployed on serverless compute are minimal. The custom services

such as legacy applications, monoliths, storage, streaming technologies, ERPs etc. that need infrastructure is a segment that we want to be very thoughtful about before prioritizing.

Pain Points

Note to reader: Several assumptions have been made in this section

We will now explore the incumbent developer journey as they go through configuring infrastructure for their Python service(s).

Developer Journey

Customer

- Starts with intent to create a Python service. Builds (including testing) the service in a development environment using appropriate platform resources.
- Write dockerfile configurations for containerization.

CI Tool

- Containerize the application using Docker and push to artifact registry.

Customer

- Determines the resource needs for the service based on NFRs such as throughput, latency, availability (SLA), cost, to name a few. Further, if infrastructure is K8s based, then determine configurations for services, such as deployments, services, ingress, security, observability, etc.
- **Write the configurations YAML for deployments, services, ingress, secrets, configs, storage, etc.**

CD Tool

- Deploy the service.

Customer

- **Verify and monitor service using observability tools.**

[Case Study Instruction] Given that the largest toil is in gluing services together (bolded above), we will now prioritize that part of the developer journey to address. We will now further decompose the pain point in the table below, using severity and frequency as the evaluation criteria. The evaluation is based on a 3 point scale.

Pain Point	Severity	Frequency	Total
Developers don't have deep knowledge of K8s (which has a steep learning curve) to properly setup their service	1	3	4
Developers are unsure how much infra resources are needed for their service resulting in under provisioning (services miss SLAs) or over provisioning (wasteful, expensive)	2	3	5

Developers spend a disproportionate amount of time to manually write configurations resulting in slower speed to market	3	3	6
--	----------	----------	----------

Given the high severity and frequency of the pain point, we will prioritize addressing manual service configuration, which will offer our customers the most leverage. The other two pain points, while important to address, can be tackled as fast followers after the initial launch. Our developer community at this time has sufficient skills and knowledge to self-service the use of infrastructure provided by the Platform team, hence the pain points in rows 1 and 2 above have lower point rating for severity compared to the pain point in row 3.

Solution Prioritization

We will list the most reasonable solutions to the pain point prioritized above, and then apply an evaluation criteria such as customer value, feasibility, and LOE.

Solution	Customer Value	Feasibility	Ease of Build	Alignment w/ Product Vision	Total
Browser based UI workflow w/ smart defaults and suggestions	1	3	2	1	7
Templated configurations	1	2	2	2	7
Intent-driven configurations using AI	3	1	1	3	8

Based on the evaluation applied above, we will prioritize building **intent-based infrastructure scaling** solutions. The intent-based configuration ranks highest for customer value generation and alignment with product vision. Customers will be able to focus their developer bandwidth on building applications used by millions of end users, and less on managing infrastructure. The complexities of K8s are abstracted away from them. The solution will help accelerate speed to market for those teams. The teams will only need to know NFRs that matter, such as throughput (RPS), latency, and the business criticality of their application which translates to availability region and other service tier requirements (SLAs). AI coding assistants will translate intent presented in natural language (NL) to infrastructure configurations.

A few additional prioritization rationale highlights:

- A UI based solution would require deep K8s knowledge.
- The template based solution may not scale well.

- The UI and template based solutions are potentially easier to build compared to an AI based solution.

Solution Approach

The invention of the transformer based architecture advanced the capability of Large Language Models (LLM) to understand intent and generate text, audio and video, with planning and reasoning abilities. These models, when combined with the power of orchestration, can be used to deploy coding assistants or agents that can in turn accelerate customer (developer) productivity.

The AI coding assistant translates customer intent to platform configurations. For this, the AI agents will need access to internal codebases, infrastructure architecture, internal tools, and other context such as governance rules, security posture, and other processes. The agent will integrate with the CI/CD pipeline aligning with existing governance and security posture. Governance processes such as evals and guardrails protect the platform, tenant services, and customers from unintended consequences.

The goal is to build these *intelligence layers* that understand developer intent and translate that into deployment configurations (YAML) that can be directly used by the platform's CI/CD and container orchestration.

Generated YAML configs are committed to team-owned Git repositories and versioned alongside application code. Once developers approve configs, the existing CI/CD pipeline handles deployment to Kubernetes clusters. The agent's role ends at config generation and Git commit.

Figure 1: A generic AI agent runtime architecture in an enterprise

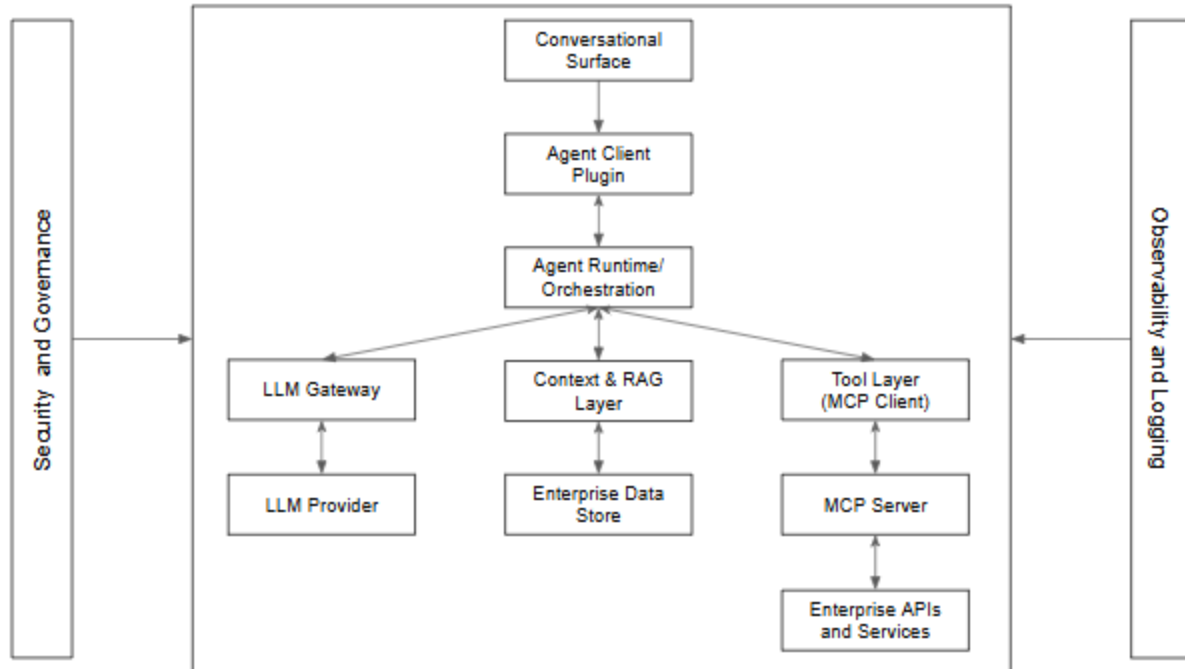
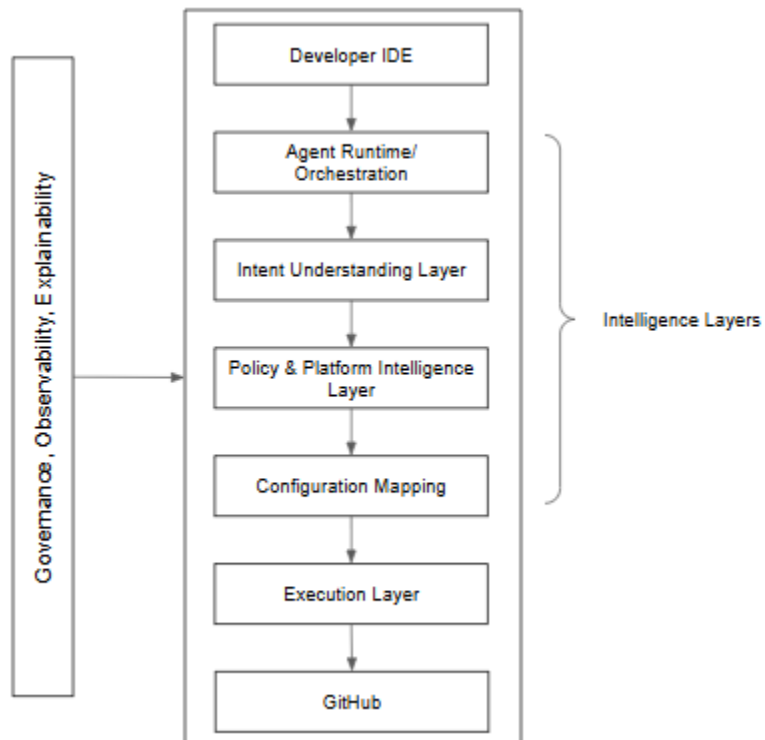


Figure 2: A generic offline RAG pipeline deployment in an enterprise



Figure 3: XYZ, Inc.'s intent-driven AI agent for platform deployments



Below we will very briefly describe what each layer of our intent-driven platform architecture is responsible for.

Layer	Description
Developer IDE	• Agent prompt surface
Agent Runtime / Orchestration	• Manages workflow • Coordinates reasoning and tool usage
Intent Understanding Layer	• Intent parser (intent model with validation and normalization) • Convert NL to JSON like spec. E.g. service, tier, RPS, region, etc.
Policy Engine	• Tier definitions (Tier-1, Tier-2, etc.) • Security & compliance rules • Cost & performance defaults • Organizational guardrails
Configuration Mapping	• Maps intent to infrastructure primitives • Generates YAML for: ◦ Deployments ◦ Services ◦ Ingress ◦ Security ◦ Storage ◦ Observability
Execution Layer	• CI/CD Pipeline • K8s for deployment orchestration
Runtime Environment / Infra	• Compute • Networking • Storage • Etc.

Build vs Buy vs Partner

Given the DevEx team's focus within the Platform organization is to increase customer deployment velocity on our infrastructure, we must be very deliberate in choosing which components and technology we want to build in-house over buying. Components we choose to build must require sufficient differentiated capabilities that are proprietary and cannot be substituted with 3rd party tools. The team's bandwidth, cost of procuring software (licensing, usage, etc.), and vendor lock-in risk are some factors to consider in making this decision.

Given we have the bandwidth of 4-5 engineers for one quarter, we assume that we have roughly 480-600 developer hours per month or a total of 1,440-1,800 developer months.[Math: ~20

working days a month. One developer has ~6 hours of capacity per day. So 120 developer hours a month per developer].

We will focus our engineering investment on the intelligence layers which involves intent parsing, policy, and configuration synthesis, while leveraging existing tools for orchestration, deployment, and observability.

The following is a list of components evaluated for build vs buy vs partner decision. Note again that the focus for the 4-5 member platform team is the intelligence layers of the stack, which is in bold in the table below..

Layer	Build / Buy / Use Existing	Rationale
Developer IDE	Assumed exists	Customers already use IDE for software development
Agent Runtime / Orchestration	Build (potentially)	This hosts workflow logic that can be built in-house using open source frameworks like self-hosted or managed LangChain/LangGraph.
Intent Understanding Layer	Buy, Build	Buy: Training a model is expensive and time consuming. Buy a medium size, high quality model. Build: Validation, standardization, schemas, etc.
Policy Engine	Build	Proprietary policies, definitions and rules
Configuration Mapping	Build	Proprietary logic to produce YAML that needs to map to platform teams execution and runtime
Execution Layer	Assumed exists	Customers already use CI/CD
Runtime Environment / Infra	Assumed exists	Customers already use the platform/infra. Potentially need to forecast resource needs for growth.
Governance, Observability, Explainability	Build	Use existing evals, guardrails and observability tools. E.g • Google DLP, AWS Guardrails • LLM-as-a-Judge • DataDog, NewRelic, Prometheus

Product Requirements

Below is a set of product requirements for core components of the intent-based intelligence layers discussed above.

Layer	High Level Product Requirements
Agent Runtime / Orchestration	• Accept NL intent from IDE plugin or CLI interface • Coordinate multi-step workflow: intent parsing, policy validation, config generation, preview, execution • Track progress, retries, partial and full failures gracefully. • Support async execution with status polling for long-running config generation • Enable rollback to previous working configuration on request • Provide interactive clarification prompts when intent is unclear (e.g., "Which region: us-east-1 or us-west-2?") • Enable tool invocation via MCP client • Enable LLM integration • Support dry-run mode that generates config without applying to infrastructure • Generate logs • Implement guardrails
Intent Understanding Layer	• Parse NL input into structured JSON spec with fields: service_name, service_type, tier, target_rps, region, sla_requirement • Validate parsed intent against required fields and return actionable error messages for missing/invalid inputs • Normalize service tier keywords (e.g., "critical" = Tier-1, "production" = Tier-2, "dev" = Tier-3) • Support both conversational input ("I need a Python API for payments handling 5k RPS") and structured input (JSON/YAML intent file) • Provide confidence score for parsed intent; flag low-confidence parses for human review • Return parsed intent to developer for confirmation before proceeding to config generation • Support iterative refinement of intent • Build guardrails for hallucination
Policy Engine	• Maintain versioned tier definitions, mapping business criticality to infrastructure settings (CPU, memory, replica count, autoscaling thresholds) • Enforce security guardrails and compliance policies • Apply cost optimization defaults such as improve bin packing of nodes, prioritize vertical auto scaling, etc. • Enforce compliance rules such requiring multi-AZ deployment for Tier-1 • Provide explainability ("why this config was chosen") • Surface policy violations with clear remediation guidance

	before config generation • Support policy override workflow requiring explicit approval from security/platform team
Configuration Mapping	• Generate configs using template-driven, deterministic approach valid for K8s to YAML for Deployment, Service, Ingress, ConfigMap, Secrets, observability • Use standard labels and annotations for cost tracking, observability, and ownership • Output YAML to version control branch for developer review before merge
General Cross Cutting NFRs	• Scalability: Support X number of total developer, Y number of total services • Throughput: Support Z peak RPS or concurrent intent requests without degradation; queue overflow returns clear error message • Latency: Intent to generate config preview in under 10 seconds (p95) • Availability: Agent available 99.5% • Correctness: Generated configs must pass K8s validation and deploy successfully ≥95% of the time • Auditability: Every config change must trace back to original intent, user, timestamp, and approval chain • Security: All LLM API calls use encryption, no sensitive data logging • Developer control: Developers can view, edit, and manually approve generated YAML before deployment. No auto-apply without explicit opt-in
Governance, Observability, Explainability	• Pre-deployment K8s validation and policy checks with remediation guidance • Build guardrails for model hallucination • Track end-to-end latency (p50, p95, p99) and alert when p95 >10s • Audit trail: log all intents, configs, approvals, deployments with user/timestamp • Monitor success rates: parse, validation, deployment; break down by error type • Inject explanation comments in YAML (e.g., "# Tier-2 = replicas: 2, platform standard") • Confidence scoring per field; flag <0.8 for human review • Track LLM costs and infrastructure costs • Thumbs up/down feedback collection after generation • Version all configs, enable rollback • Alert on incident-causing configs

Roadmap

	Month 1	Month 3	Month 6
GTM Phase	MVP	Open Beta	GA
Business Outcome	Understand NL deployment intent	Reduce deployment toil	Reduce deployment toil
Success Criteria	Produce an output JSON schema with high level deployment primitives	Reduce the number of manually written config by >50%, WoW Scale to support at ~20% of the TAM	Reduce the number of manually written config by >90%, WoW Scale to support >80% of TAM NPS of >70
In Scope	Intent Understanding Layer Limited observability for DevEx team only	All layers of the stack Limited support	All layers of the stack CLI, SDK
Out of scope	Other layers of the intelligence stack Customer observability	Full adherence to SLA defined in requirements	

Our approach to crafting the above roadmap is shown below. We list the components and evaluate the level of effort for each layer, followed by describing each phase of the launch.

Layer	LOE (Time and Complexity)
Agent Runtime / Orchestration	<u>HIGH</u> Workflow coordination, memory and state management, async execution, retry logic, integration points with IDE and Git, streaming responses, error handling across layers
Intent Understanding Layer	<u>LOW</u> Managed inference w/ 3rd party model, thin validation/normalization wrapper around it
Policy Engine	<u>MEDIUM</u> Encoding tier definitions, security rules, defaults, compliance guardrails, versioning rules, dependency rules, etc.

Configuration Mapping	<u>HIGH</u> Translating high level JSON intent to multiple YAML types (deployment, service, ingress, ConfigMap, secrets), handle region-specific defaults, generate syntactically correct K8s primitives, managing dependencies between resources
Governance, Observability, and Explainability	<u>MEDIUM</u>

MVP - Launch in 1 month w/ 4-5 engineers

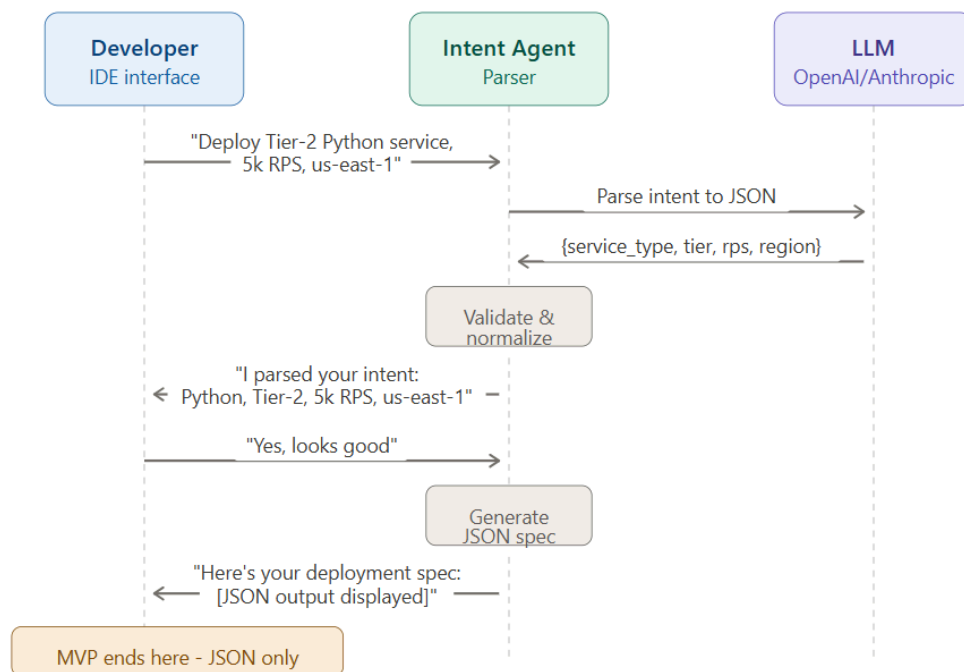
Golden path, templated solutions have already proved their value (common practice in enterprises) so repeating this solution will offer no incremental learning opportunity. Building a full intelligent stack is not suitable for experimentation purpose.

The highest leverage validation is confirmation that the system can understand intent presented in NL and convert it to valid structured and normalized schema.

Given that the LOE for this layer is LOW, we have high confidence that the squad of engineers will be able to deliver this on time. The requirements for this layer is described in the [Product Requirements](#) section of this document.

To ensure that we have supportive customer partnership, we will enlist one customer team/developer to run this validation exercise with.

Figure 4: Developer Interaction w/ Intent Agent



Open Beta - Launch in 3 months

In this phase, the team will build out the rest of the layers of the solution. Any customer in need of deploying a containerized microservice will be able to use the solution. We will provide best effort production support (applicable to the intelligence layers). We will build out internal product and operational analytics/observability.

GA - Launch in 6 months

This phase signifies the stable launch of the intelligence layers. The solution will be available broadly to all teams at XYZ, with full production support and SLA.

There is an opportunity here to explore if the agentic solution is able to learn and apply deployment strategies from other similar services previously running successfully in production. This is an area we can explore provided there is engineering bandwidth.

GTM Strategy

Our go-to-market strategy will use many or all of the following tactics

- Awareness Oriented
 - Customer outreach: utilize existing customer list and POC
 - Announcements at internal conferences: Short demo
 - Monthly newsletter: "Meet the Intent Agent"
 - Slack channel: #agentic-deployment for questions, updates, and feedback
 - Lunch-and-learns: Host live sessions, record and share
 - Publish internal blog posts: "How mobile deployed 3 services in 1 day using Intent Agent"
 - Office hours: Weekly 30-min drop-in sessions for teams trying the tool for the first time
- Scale Oriented
 - Onboarding docs and KB articles
 - Regular training sessions
 - Integration into new hire onboarding
 - Success metrics dashboard
 - Migration playbook
 - Executive sponsorship

FAQ

1. How do you determine which part of the path to production is the most broken and causing the most pain for customers?

Addressed in the [Problem Statement](#) section.

2. What data or signals would you use to quantify this toil; and what would success look like for our developers? (Frame and share your assumptions if any)

Addressed in the [Problem Statement](#) section.

3. How would you validate your assumptions with our internal engineers without being a distraction to their sprint? If this AI layer is successful, how will the organization change? Define at least 2–3 North Star metrics.

We will observe CI/CD logs, Slack channels, PRs, lightweight async engagement, targeted 1:1s with developers, and partner with the pilot team's tech lead for a couple of hours per week over 4 weeks.

If successful, deployment shifts from power users requiring K8s expertise to a natural language intent based experience available to all engineers. The DevEx team will no longer be a bottleneck. Developers onboard faster without deep understanding of K8s or infra.

North Star Metrics: Addressed in the [North Star Metrics](#) section.

4. Propose & describe a single interface or agentic flow that acts as a translation layer for the customer's intent (You can choose and mention the layers you are targeting and assumptions you are making, but justify your choice)

Addressed in the [Solution Approach](#) section.

5. How would you prioritize which phase of the lifecycle or service to automate first? Show us the logic behind what you choose to build vs. what you choose to delay.

Address across many sections in this doc, but primarily in the [Roadmap](#) section and [Build vs Buy vs Partner](#) section.

6. How do you connect developer value (speed) with business results (stability/cost)?

Cost reduction: Platform-enforced resource sizing and defaults prevent over-provisioning.

Speed to market: Reduced deployment time reclaims engineering capacity for customer-facing feature development

Developer velocity: Eliminates K8s as prerequisite skill, reduces onboarding time, improves developer satisfaction and retention

Stability through consistency: Platform-standard security, observability, and resilience configs applied by default reduce configuration errors that cause production incidents

7. (Optional) - Use an AI tool (v0, Mermaid, etc.) to show us the new developer interface. How does the solution change the developer's "Tuesday afternoon"?

Addressed in the [MVP](#) section.

8. How do you move from a vision to a testable MVP? What is the quickest way to learn if your solution actually solves the problem? Describe a MVP that could provide value to a single team within 4 weeks.

Addressed in the [MVP](#) section.

9. How do you build trust with skeptical internal customers? How do you design the product to provide transparency and control so senior engineers trust the AI?

Addressed by the Governance, Observability, and Explainability layer across the doc.

10. What is your approach to marketing this platform internally to ensure adoption?

Addressed in the [GTM Strategy](#) section.

APPENDIX

Exhibit A

Which type of services are more used in large enterprises, containerized or serverless?

Short answer: **containerized services are still more widely used in large enterprises than serverless**, but both are heavily used—and often together.

Exhibit B



Intent-Based Developer Platform by Vi...
PDF

Review the Product Requirements section of the uploaded PDF and help me refine that section only. Prepare a plan before you proceed.

Here's the plan I'll follow to refine **only the Product Requirements** section of your PDF.

Exhibit C

Mermaid code for components of AI agent created by ChatGPT
flowchart TD

%% Client Layer

A[Developer IDE] --> B[Agent Client Plugin]

%% Core Orchestrator

B --> C[Agent Runtime / Orchestrator]

%% Core Dependencies

C --> D[LLM Gateway]

C --> E[Context / RAG Layer]

C --> F[Tool Layer (MCP Client)]

%% LLM Side

D --> G[LLM Providers]

%% RAG Side

E --> H[Enterprise Data Sources]

%% Tooling via MCP

F --> I[MCP Servers]

I --> J[External APIs / Systems]

%% Streaming back

C --> B

%% Optional Observability / Security (side concerns)

C -. -> K[Observability & Logging]

C -. -> L[Security & Governance]

D -. -> K

F -. -> L

Exhibit D

Validate service_type

valid_types = ["python-fastapi", "python-flask", "nodejs-express", "java-spring"]

assert parsed["service_type"] in valid_types ✓

Validate tier

valid_tiers = ["tier-1", "tier-2", "tier-3"]

assert parsed["tier"] in valid_tiers ✓

Validate regions

```
valid_regions = ["us-east-1", "us-west-2", "eu-west-1", "ap-southeast-1"]  
for region in parsed["regions"]:  
    assert region in valid_regions ✓
```

```
# Validate RPS range  
assert 0 < parsed["target_rps"] < 100000 ✓
```

```
# Check for required fields  
required_fields = ["service_name", "service_type", "tier", "target_rps"]  
for field in required_fields:  
    assert field in parsed ✓
```